BAKERY SALES SQL PROJECT

Noah Wamble

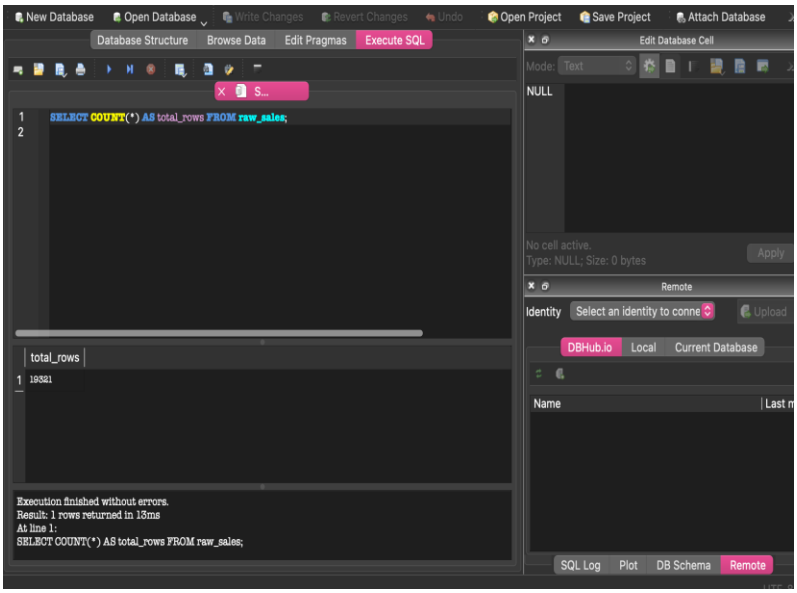
Objective: Find which products, stores and days waste the most money and recommend where production should change.

Synthetic dataset from Kaggle. Data is in Rupees (₹). (Going off of data).

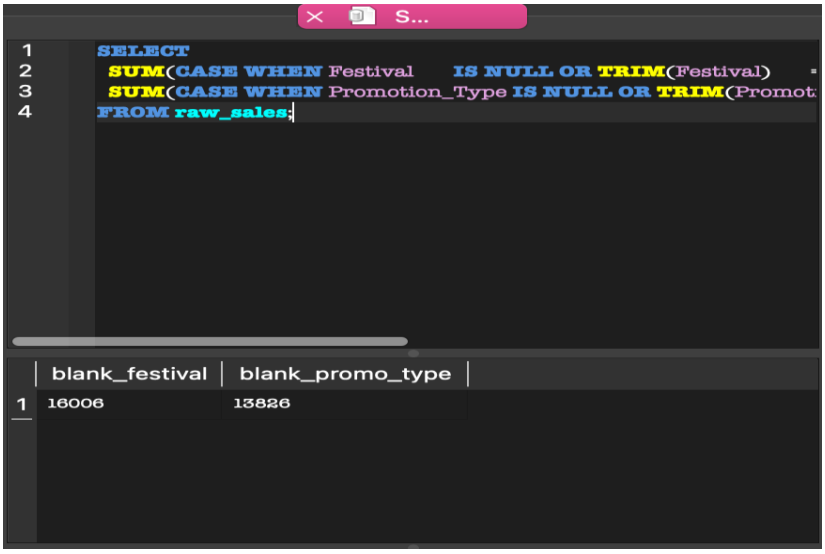
We need to clean and check data, check calculations, find logic problems, make a new table with clean data, build a daily inventory table, check clean data, and finally make our conclusions.

Part 1: Clean up and check data

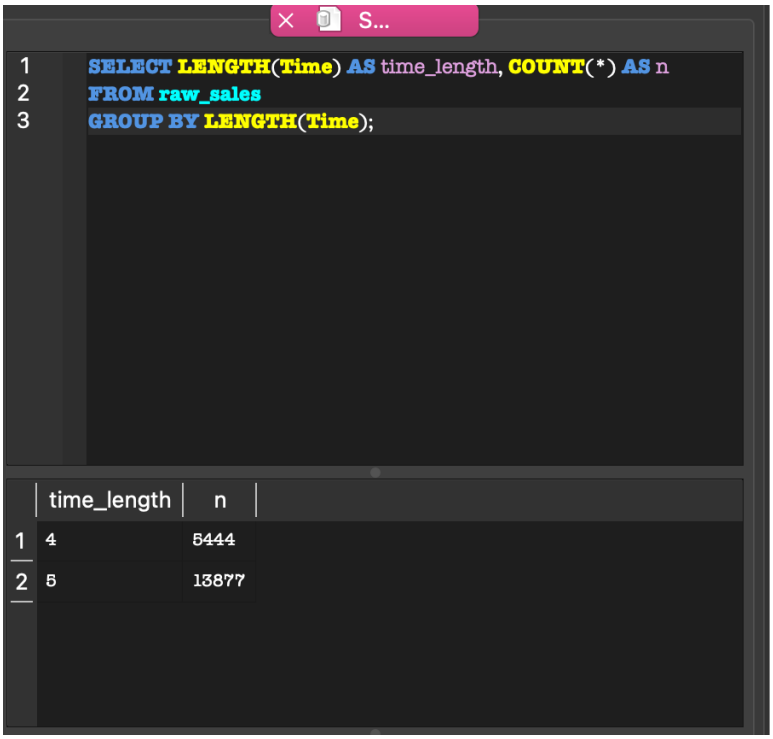
Check rows



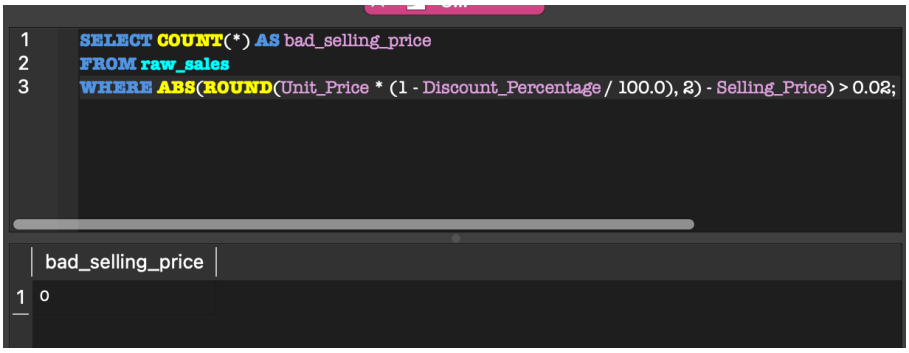
Check blanks per column



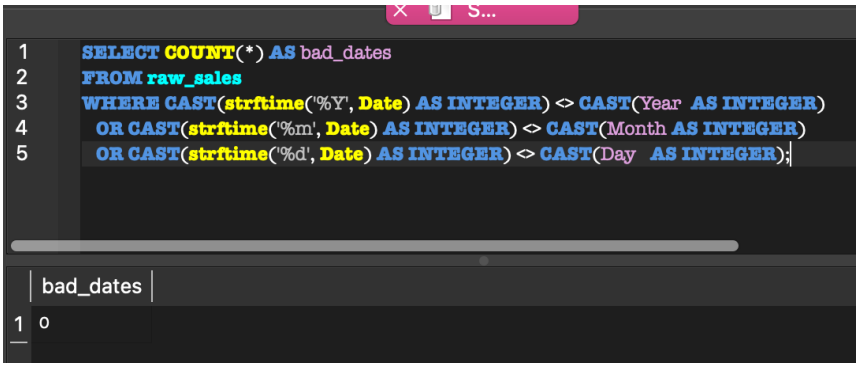
Look at time format



Part 2: Make sure calculations still align with numbers already in the table.



Check for bad dates



Part 3: Find Logic Problems

```sql
SELECT COUNT(*) AS discount_without_promo,
    ROUND(AVG(CAST(Discount_Percentage AS REAL)), 1) AS avg_discount_pct
FROM raw_sales
WHERE Promotion_Applied = 'No'
 AND CAST(Discount_Percentage AS REAL) > 0;
```

	discount_without_promo	avg_discount_pct
1	2089	5.2

I see that there is an error in the data. Maybe employees forgot to list a promotion when selling products with a discount. The average discount rate is 5.2%

I also looked at where the bakery lost money on specific orders. Six out of the 7 results are Milk with around a 28% discount.

```sql
SELECT Order_ID, Product, Unit_Price, Cost_Price, Discount_Percentage, Selling_Price, Profit
FROM raw_sales
WHERE CAST(Profit AS REAL) < 0;
```

	Order_ID	Product	Unit_Price	Cost_Price	Discount_Percentage	Selling_Price	Profit
1	ORD00656	Milk	30.8	22.41	27.4	22.36	-0.1
2	ORD00872	Milk	30.69	22.46	28.6	21.91	-0.55
3	ORD00892	Milk	29.57	21.41	29.2	20.94	-0.47
4	ORD01895	Milk	30.71	22.34	27.7	22.2	-0.14
5	ORD02108	Paneer	80.71	58.78	27.4	58.6	-0.18
6	ORD02474	Milk	30.98	21.79	29.9	21.72	-0.21
7	ORD03510	Milk	31.25	22.62	28.4	22.38	-0.24

Let's check to see if units sold are per order by checking for any repeated orders.

```sql
SELECT Date, Store_ID, Product,
    COUNT(*) AS order_lines,
    SUM(CAST(Quantity AS INTEGER))     AS total_qty_ordered,
    MIN(CAST(Units_Sold AS INTEGER))   AS units_sold_min,
    MAX(CAST(Units_Sold AS INTEGER))   AS units_sold_max,
    MIN(CAST(Units_Produced AS INTEGER)) AS produced_min,
    MAX(CAST(Units_Produced AS INTEGER)) AS produced_max
FROM raw_sales
GROUP BY Date, Store_ID, Product
```

	Date	Store_ID	Product	order_lines	total_qty_ordered	units_sold_min	units_sold_max	produced_min	produced_max
1	2023-01-01	ST01	Bread	3	7	7	7	8	8
2	2023-01-01	ST01	Chocolate Cake	2	2	2	2	3	3
3	2023-01-01	ST01	Coffee Powder	4	14	14	14	16	22
4	2023-01-01	ST01	Honey	2	3	3	3	4	4
5	2023-01-01	ST01	Muffin	4	15	15	15	17	22
6	2023-01-01	ST01	Tea Powder	2	4	4	4	5	5
7	2023-01-01	ST02	Cookies	2	9	9	9	10	13
8	2023-01-01	ST02	Nankhatai	2	14	14	14	19	20
9	2023-01-01	ST03	Bread	3	8	8	8	9	9
10	2023-01-01	ST03	Chocolate Cake	2	2	2	2	3	3

We understand that units sold does not equal order numbers. As we can see bread on the first row has three orders but 7 actually sold. This can help us build a more accurate calculation now as the actual number of units sold and profit is going to be different.

Now we need to look at production numbers that don't agree with each other.

```
SELECT COUNT(*) AS conflicting_groups
FROM (
  SELECT Date, Store_ID, Product
  FROM raw_sales
  GROUP BY Date, Store_ID, Product
  HAVING MIN(CAST(Units_Produced AS INTEGER)) <> MAX(CAST(Units_Produced AS INTEGER))
);
```

conflicting_groups
549

We have 549 conflicting lines that we will need to fix. We don't want and can't understand having different numbers for units produced.

Part 4: Make New Table and Clean Up Data

Create a new table called "sales_clean".

Add 8 columns to start the table.

```
SELECT
  rowid                AS raw_id,
  TRIM(Order_ID)       AS order_id,
  TRIM(Customer_ID)    AS customer_id,
  TRIM(Store_ID)       AS store_id,
  CAST(Customer_Age AS INTEGER)  AS customer_age,
  Customer_Gender      AS customer_gender,
  Customer_Segment     AS customer_segment,
  CASE WHEN Loyalty_Member = 'Yes' THEN 1 ELSE 0 END AS is_loyalty_member
FROM raw_sales;
```

	raw_id	order_id	customer_id	store_id	customer_age	customer_gender	customer_segment	is_loyalty_member
1	1	ORD00001	CUST0328	ST02	46	Other	Young Professional	1
2	2	ORD00001	CUST0328	ST02	46	Other	Young Professional	1
3	3	ORD00002	CUST0016	ST03	21	Female	Student	0
4	4	ORD00002	CUST0016	ST03	21	Female	Student	0
5	5	ORD00002	CUST0016	ST03	21	Female	Student	0
6	6	ORD00003	CUST0389	ST02	35	Male	Young Professional	0
7	7	ORD00004	CUST0053	ST02	23	Male	Student	0
8	8	ORD00004	CUST0053	ST02	23	Male	Student	0
9	9	ORD00005	CUST0454	ST01	38	Male	Family	0
10	10	ORD00005	CUST0454	ST01	38	Male	Family	0
11	11	ORD00005	CUST0454	ST01	38	Male	Family	0

Add more rows including a time of day row and fix time of day numbers.

```sql
DROP TABLE IF EXISTS temp_sales;

CREATE TABLE temp_sales AS
SELECT
 s.*,
 DATE(r.Date)                AS order_date,
 CAST(r.Year  AS INTEGER)        AS year,
 CAST(r.Month AS INTEGER)        AS month,
 strftime('%Y-%m', r.Date)       AS year_month,
 r.Day_of_Week               AS day_of_week,
 CASE WHEN r.Weekend = 'Yes' THEN 1 ELSE 0 END AS is_weekend,
 CAST(r.Hour AS INTEGER)         AS hour,
 SUBSTR('0' || TRIM(r.Time), -5)  AS order_time,
 CASE
  WHEN CAST(r.Hour AS INTEGER) < 11 THEN 'Morning'
  WHEN CAST(r.Hour AS INTEGER) < 15 THEN 'Midday'
  WHEN CAST(r.Hour AS INTEGER) < 18 THEN 'Afternoon'
  ELSE 'Evening'
 END                         AS daypart
FROM sales_clean s
JOIN raw_sales r ON r.rowid = s.raw_id;

DROP TABLE sales_clean;
ALTER TABLE temp_sales RENAME TO sales_clean;
```

order_date	year	month	year_month	day_of_week	is_weekend	hour	order_time	daypart
2023-01-01	2023	1	2023-01	Sunday	1	8	08:47	Morning
2023-01-01	2023	1	2023-01	Sunday	1	8	08:47	Morning
2023-01-01	2023	1	2023-01	Sunday	1	15	15:14	Afternoon
2023-01-01	2023	1	2023-01	Sunday	1	15	15:14	Afternoon
2023-01-01	2023	1	2023-01	Sunday	1	15	15:14	Afternoon
2023-01-01	2023	1	2023-01	Sunday	1	15	15:44	Afternoon
2023-01-01	2023	1	2023-01	Sunday	1	18	18:06	Evening
2023-01-01	2023	1	2023-01	Sunday	1	18	18:06	Evening
2023-01-01	2023	1	2023-01	Sunday	1	14	14:36	Midday
2023-01-01	2023	1	2023-01	Sunday	1	14	14:36	Midday
2023-01-01	2023	1	2023-01	Sunday	1	14	14:36	Midday

I want to change the festival day from blank to "None". A large majority of rows have no festival and we don't want to delete them. Also added a column to tell if its festival day or not which can help later on using 1 and 0.

```sql
DROP TABLE IF EXISTS temp_sales;

CREATE TABLE temp_sales AS
SELECT
 s.*,
 r.Season                        AS season,
 COALESCE(NULLIF(TRIM(r.Festival), ''), 'None')  AS festival,
 CASE WHEN NULLIF(TRIM(r.Festival), '') IS NULL THEN 0 ELSE 1 END AS is_festival_day,
 r.Weather                       AS weather,
 CAST(r.Temperature AS REAL)             AS temperature_c
FROM sales_clean s
JOIN raw_sales r ON r.rowid = s.raw_id;

DROP TABLE sales_clean;
ALTER TABLE temp_sales RENAME TO sales_clean;
```

Next we convert the numbers to real numbers for better calculations.

```sql
DROP TABLE IF EXISTS temp_sales;

CREATE TABLE temp_sales AS
SELECT
 s.*,
 r.Product                    AS product,
 r.Category                   AS category,
 r.Storage_Type                 AS storage_type,
 r.Perishability              AS perishability,
 CAST(r.Quantity         AS INTEGER) AS quantity,
 CAST(r.Unit_Price       AS REAL)    AS unit_price,
 CAST(r.Cost_Price       AS REAL)    AS cost_price,
 CAST(r.Discount_Percentage AS REAL)    AS discount_pct,
 CAST(r.Selling_Price      AS REAL)    AS selling_price,
 CAST(r.Total_Bill       AS REAL)    AS line_revenue,
 CAST(r.Profit           AS REAL)    AS line_profit,
 ROUND(CAST(r.Cost_Price AS REAL) * CAST(r.Quantity AS INTEGER), 2) AS line_cost,
 r.Payment_Method               AS payment_method
FROM sales_clean s
JOIN raw_sales r ON r.rowid = s.raw_id;

DROP TABLE sales_clean;
ALTER TABLE temp_sales RENAME TO sales_clean;
```

Add promotion columns and sort rows into three different groups like “Full price”, “Unlabeled discount”.

```sql
DROP TABLE IF EXISTS temp_sales;

CREATE TABLE temp_sales AS
SELECT
 s.*,
 CASE WHEN r.Promotion_Applied = 'Yes' THEN 1 ELSE 0 END AS is_promotion,
 COALESCE(NULLIF(TRIM(r.Promotion_Type), ''), 'None')    AS promotion_type,
 CASE
  WHEN r.Promotion_Applied = 'Yes'          THEN 'Promotion'
  WHEN CAST(r.Discount_Percentage AS REAL) > 0 THEN 'Unlabeled discount'
  ELSE 'Full price'
 END                                       AS discount_type,
 CAST(r.Customer_Rating AS INTEGER)                AS customer_rating,
 CASE WHEN CAST(r.Profit AS REAL) < 0      THEN 1 ELSE 0 END AS flag_sold_below_cost,
 CASE WHEN CAST(r.Quantity AS INTEGER) > 10 THEN 1 ELSE 0 END AS flag_bulk_quantity
FROM sales_clean s
JOIN raw_sales r ON r.rowid = s.raw_id;

DROP TABLE sales_clean;
ALTER TABLE temp_sales RENAME TO sales_clean;
```

Check total revenue.

```sql
1  SELECT ROUND(SUM(line_revenue), 2) AS total_revenue FROM sales_clean;
```

	total_revenue
1	2690806.05

Part 5: Build Daily Inventory Table

Create the daily inventory table

```sql
CREATE TABLE daily_inventory AS
SELECT
 DATE(Date)          AS inv_date,
 strftime('%Y-%m', Date)  AS year_month,
 Store_ID            AS store_id,
 Product             AS product,
 Category            AS category,
 Perishability       AS perishability,
 MAX(CAST(Units_Produced AS INTEGER)) AS units_produced,
 MAX(CAST(Units_Sold    AS INTEGER)) AS units_sold,
 MAX(CAST(Units_Produced AS INTEGER)) - MAX(CAST(Units_Sold AS INTEGER)) AS units_unsold,
 ROUND(100.0 * (MAX(CAST(Units_Produced AS INTEGER)) - MAX(CAST(Units_Sold AS INTEGER)))
     / MAX(CAST(Units_Produced AS INTEGER)), 1) AS waste_pct,
 ROUND(AVG(CAST(Cost_Price AS REAL)), 2) AS avg_unit_cost,
 CASE WHEN MIN(CAST(Units_Produced AS INTEGER)) <> MAX(CAST(Units_Produced AS INTEGER))
    THEN 1 ELSE 0 END AS flag_conflicting_production
FROM raw_sales
GROUP BY DATE(Date), Store_ID, Product;
```

Create a new column for waste cost and create the formula.

```sql
ALTER TABLE daily_inventory ADD COLUMN waste_cost REAL;

UPDATE daily_inventory
SET waste_cost = ROUND(units_unsold * avg_unit_cost, 2);
```

Part 6: Check Clean Data

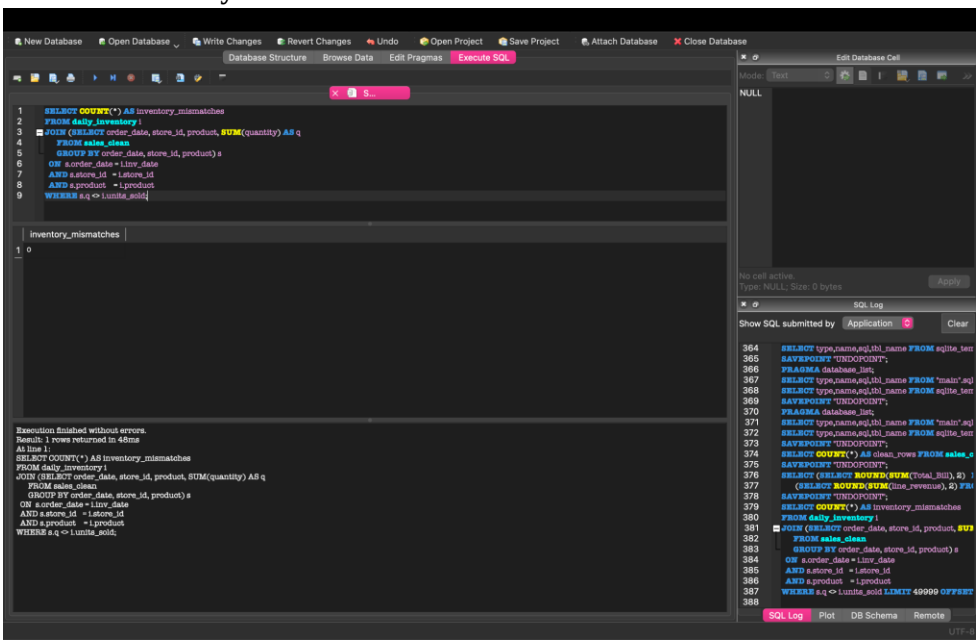
Check revenue from both tables to make sure no lost data.

```
SELECT (SELECT ROUND(SUM(Total_Bill), 2) FROM raw_sales) AS raw_revenue,
    (SELECT ROUND(SUM(line_revenue), 2) FROM sales_clean) AS clean_revenue;
```

raw_revenue	clean_revenue
2690806.05	2690806.05

We can confirm that no data has been lost.

Check if inventory matches the order totals



It shows 0 inventory mismatches.

Conclusion

Make a small table to show each problem and how many rows were affected.

```sql
SELECT 'Unlabeled discounts' AS issue, COUNT(*) AS n_rows FROM sales_clean WHERE discount_type = 'Unlabeled discount'
UNION ALL
SELECT 'Sold below cost',          COUNT(*)       FROM sales_clean WHERE flag_sold_below_cost = 1
UNION ALL
SELECT 'Bulk quantity (>10)',      COUNT(*)       FROM sales_clean WHERE flag_bulk_quantity = 1
UNION ALL
SELECT 'Conflicting production',   COUNT(*)       FROM daily_inventory WHERE flag_conflicting_production = 1;
```

	issue	n_rows
1	Unlabeled discounts	2089
2	Sold below cost	7
3	Bulk quantity (>10)	71
4	Conflicting production	549

Look at units produced and look at how much waste the bakery has.

```sql
-- Overall waste vs. profit
SELECT SUM(units_produced) AS produced,
    SUM(units_unsold)  AS unsold,
    ROUND(100.0 * SUM(units_unsold) / SUM(units_produced), 1) AS waste_pct,
    ROUND(SUM(waste_cost), 2) AS waste_cost,
    (SELECT ROUND(SUM(line_profit), 2) FROM sales_clean) AS total_profit
FROM daily_inventory;
```

produced	unsold	waste_pct	waste_cost	total_profit
61952	18925	30.5	894343.6	1143737.55

We see 61952 units produced but have 18925 units unsold showing that there is a 30.5% waste percentage.

Let's look at waste percentage by each category of the bakery.

```sql
-- Waste by category
SELECT category,
    ROUND(SUM(waste_cost), 2) AS waste_cost,
    ROUND(100.0 * SUM(waste_cost) / (SELECT SUM(waste_cost) FROM daily_inventory), 1) AS share_pct,
    ROUND(100.0 * SUM(units_unsold) / SUM(units_produced), 1) AS waste_pct
FROM daily_inventory
GROUP BY category
ORDER BY waste_cost DESC;
```

	category	waste_cost	share_pct	waste_pct
1	Cakes	418323.84	46.8	41.8
2	Dairy & Spreads	155557.83	17.4	35.2
3	Indian Bakery	119403.4	13.4	27.1
4	Beverages	73840.34	8.3	36.2
5	Pastry & Sweet	69949.05	7.8	31.2
6	Bread & Daily	45261.64	5.1	26.0
7	Savoury Bakery	12007.5	1.3	20.4

Look at profit margins for full price and discounted sales.

```sql
-- Promotions vs. full price
SELECT discount_type,
    ROUND(100.0 * SUM(line_profit) / SUM(line_revenue), 1) AS margin_pct,
    ROUND(AVG(quantity), 2) AS avg_qty
FROM sales_clean
GROUP BY discount_type;
```

	discount_type	margin_pct	avg_qty
1	Full price	45.4	2.15
2	Promotion	36.7	2.42
3	Unlabeled discount	42.3	2.15

Items sold at full price give the bakery a 45.4% profit margin vs. the promotion profit margin being 36.7%

Key Findings and Recommendations

The bakery's biggest money problem is its overproduction. Over two years, the three stores produced 61,952 units and 30.5% of them (18,925 units) went unsold. The cost of making that unsold product was ₹894,344, about 78% of the bakery's total profit of ₹1,143,738 for the same period.

Key Findings

1. Cake is almost half of all the waste. Cakes are 46% of waste and also 42% of cakes go unsold.
2. All three stores share the same issue so that means that its not a specific stores fault but instead falls on the production planning.
3. Using promotions and discounts is more costly than what they bring in. Promoted goods create a 36% profit margin compared to the 45% at full price and customers only buy around 13% more goods per product line.

Recommendations

1. Move cakes to made-to-order and have less ready to go cakes on display. Cutting cake waste in half would result in around **₹200,000** saved over 2 years.
2. Use discounts to clear stock and during holiday times instead of using it to drive sales. Using discounts on perishable items can help recover some of the money lost on waste products.
3. Improve how data is logged because throughout the 549 days, stores all had conflicting numbers showing. With new changes we need to see if these changes actually work with accurate data.